American International University- Bangladesh

Department of Electrical and Electronic Engineering

EEE 4103: Microprocessor and Embedded Systems Laboratory

Title: Familiarization with an STM32 microcontroller board, studying an LED blink test, and implementing a 2-push button LED activation circuit using an STM32 microcontroller board.

Introduction:

The objectives of this experiment are to

1. Study the STM32 Microcontroller Board.
2. Learn basic programming commands of the STM32 Microcontroller Board.
3. Apply the coding techniques of the STM32 Microcontroller Board.
4. Implement the LED blink test using the STM32 Board.
5. Develop a circuit that includes two pushbuttons and two LEDs to control two LEDs separately by ensuring proper connections for efficient functionality.
6. Develop a simulation circuit model using Proteus incorporating an LED blink test and controlling two LEDs with two pushbuttons separately.

Theory and Methodology:

STM32CubeIDE is an advanced C/C++ development platform with peripheral configuration, code generation, code compilation, and debug features for the STM32 microcontrollers and microprocessors. STM32CubeIDE includes build and stack analyzers that provide the user with useful information about project status and memory requirements. STM32CubeIDE includes standard and advanced debugging features including views of CPU core registers, memories, and peripheral registers, a live variable watch, a serial wire viewer interface, or a fault analyzer.

Overview of STM32 Nucleo-F401RE Board:

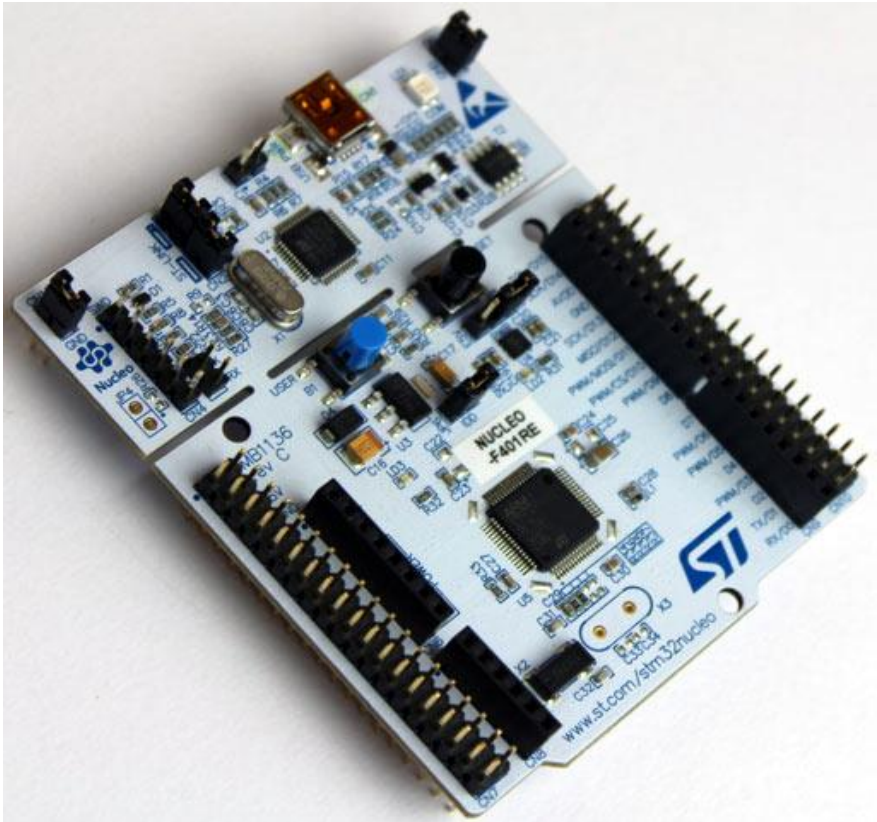

Fig. 1 STM32 Nucleo-F401RE Microcontroller Board.

© Dept. of EEE, Faculty of Engineering, American International University-Bangladesh (AIUB)

Pin Configuration:

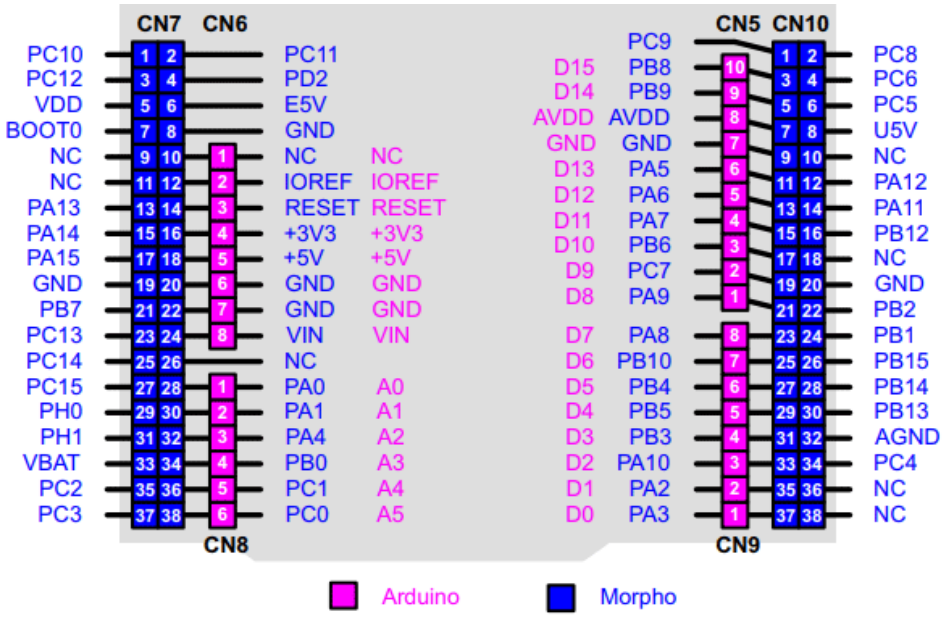

Fig. 2 Pin Configuration of the STM32 Nucleo-F401RE Microcontroller Board.

Pin Configuration from STM32Cube IDE:

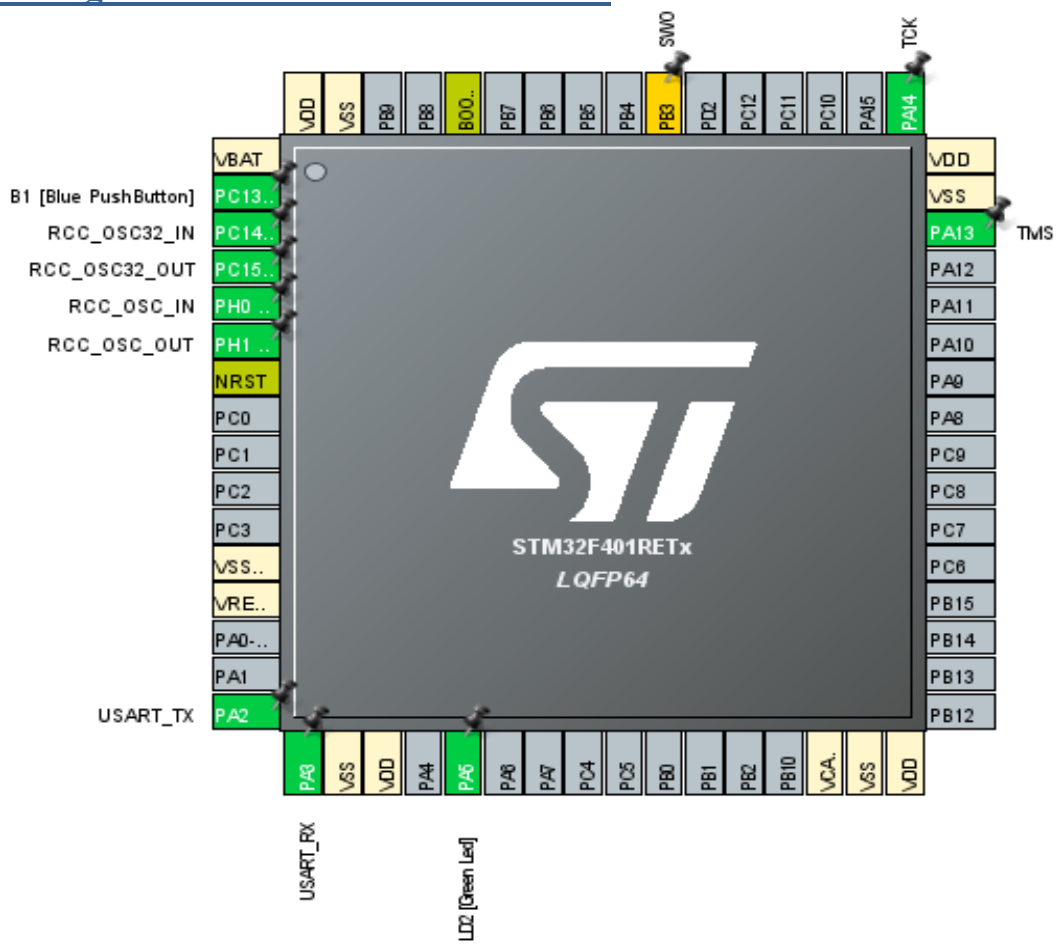

Fig. 3 Pin Configuration of the STM32 Nucleo-F401RE from the STM32Cube IDE.

Apparatus:

1) STM32 Cube IDE (1.0.1 or any recent version)
2) STM32 Microcontroller board
3) Two LED lights (Red and Green)

4) Two 100 Ω resistors and two 10kW resistors
5) Two push button switches
6) Jumper wires

Experimental Procedure:

The main task of this experiment is to implement two circuits. One circuit will show an LED blinking test, and another one will show how 2 push buttons can control 2 LEDs separately. For this purpose, we need to do it in the following three stages:
In the first stage, we will configure the system by connecting the STM32 Nucleo board to the PC's USB terminal as shown in Fig. 4 (a) and opening the STM32Cube IDE as in Fig. 4 (b).

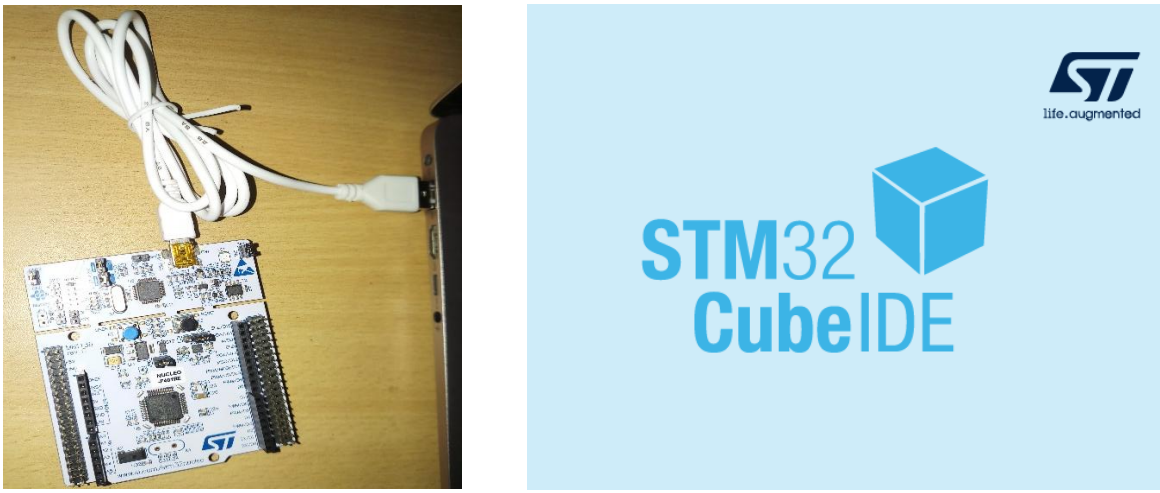

Fig. 4 (a) Connection diagram of the STM32 Nucleo-F401RE Board with the PC's USB port and (b) STM32Cube IDE program launcher

After that, we will write the C/C++ code inside the while loop of the program generated using the build project command from the STM32Cube IDE.
Then we will generate the hex code, which is required to upload into the STM32F401RE programmer board in real-time hardware to test its functionality as shown in Fig. 5. The generated hex file is required to simulate the circuit in the Proteus simulation software.

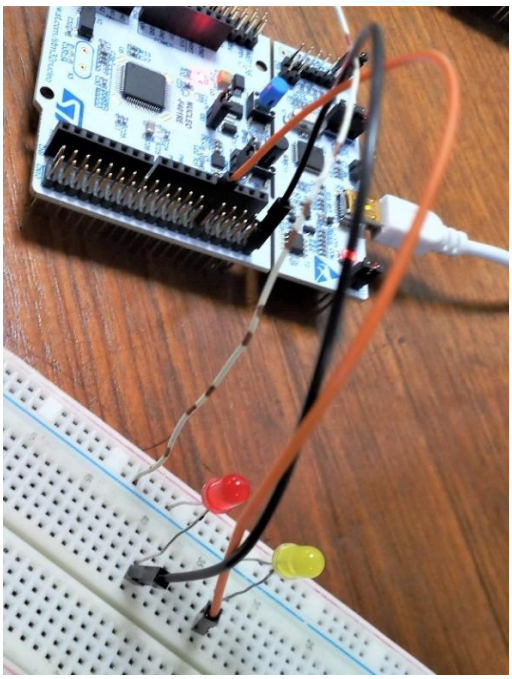

Fig. 5 Connection diagram of the LEDs with the STM32 Nucleo-F401RE Board.

Using STM32Cube IDE to write code for a simple LED blink program:

1. Open the STM32Cube IDE and in the first while(1){} loop write the program as in Fig. 6. Here, the HAL_GPIO_TogglePin() command toggles the signal level between HIGH and LOW levels sent to the I/O pin 5 from the STM32 microcontroller.

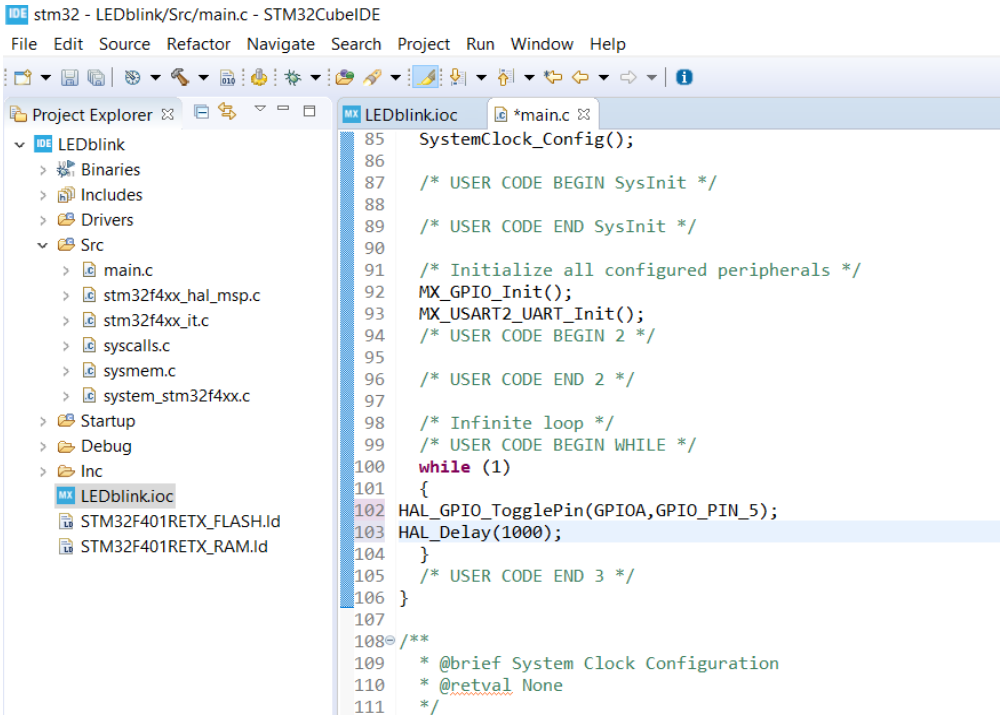

Fig. 6 Writing the program in the while(1){} loop.

2. After that, save the project file and then build the project as shown in Fig. 7.

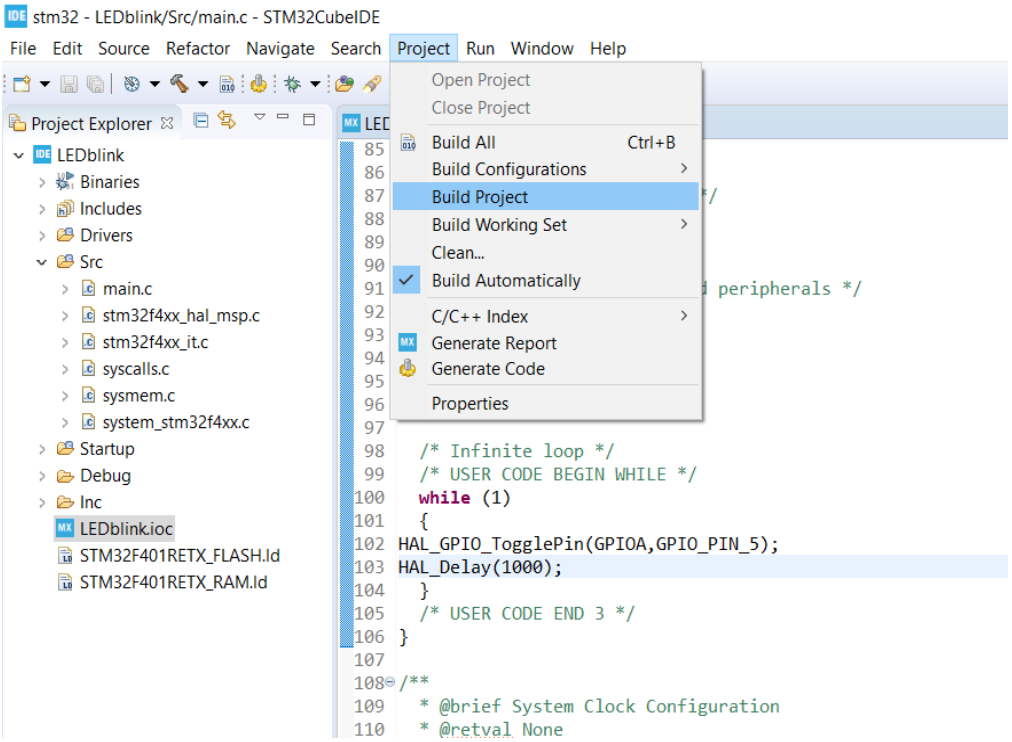

Fig. 7 Saving the program written in the while(1){} loop.

3. Next, run the program to debug as STM32 MCU C/C++ Application as shown in Fig. 8.

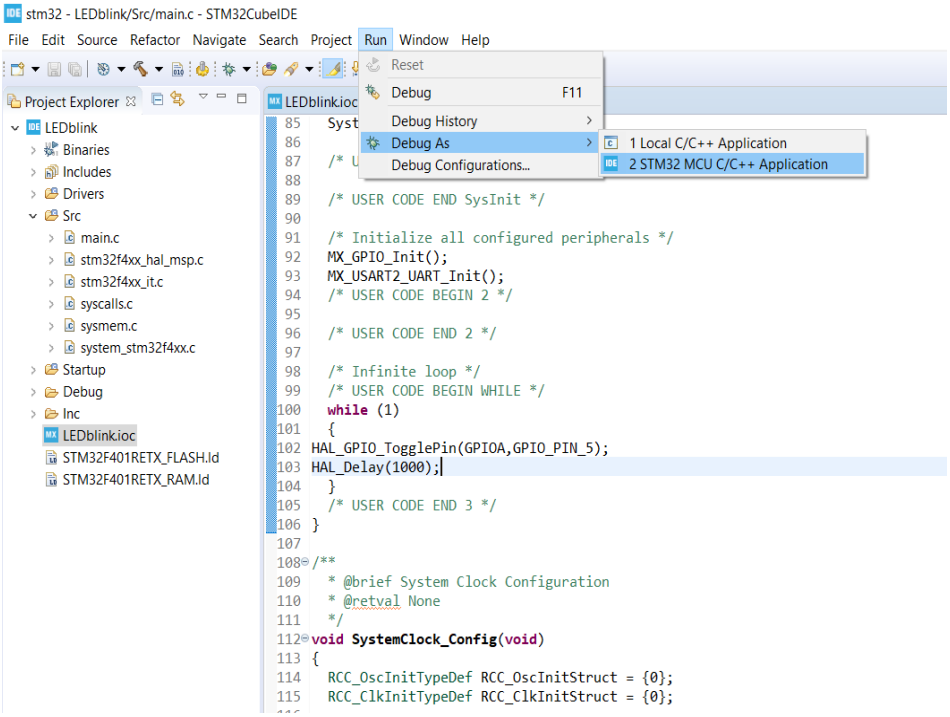

Fig. 8 Running the program written in the while(1){} loop.

4. Click on the resume option or press the function button (F8) to see that the board is showing the results of blinking the LED light.

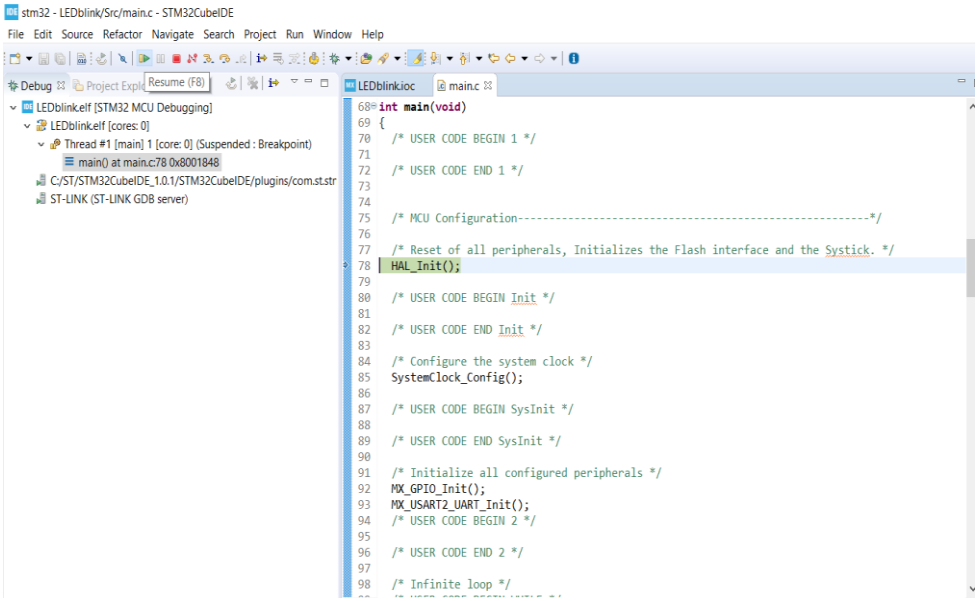

Fig. 9 Resuming the program written in the while(1){} loop.

Now, try to rewrite the code for the traffic light control system

Here, the HAL_GPIO_WritePin command may be used instead of TogglePin command to send the HIGH- and LOW-level signals by the GPIO_PIN_SET or GPIO_PIN_RESET commands respectively to the I/O port pins as the output signals from the STM32 microcontroller; and HAL_GPIO_ReadPin command may be used instead of TogglePin command to receive the HIGH- and LOW-level signals from the I/O port pins as the input signals. To apply a delay to a pin for a signal level, you can use the HAL_Delay(delay duration in ms) command with the variable names for the delay parameter meaning the duration of the delay. Besides, the "*if else*" command may be used to generate conditional statements to check the switch status at the input ports of the STM32 board.

Copy and paste the following codes inside the while(1){} loop.

```
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == 0) {
        // Button 1 is pressed, turn on the Green LED and turn off the Red LED
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); // Green LED ON
        }
else {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // Green LED OFF
        }
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == 0) {
        // Button 1 is pressed, turn on the Green LED and turn off the Red LED
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); // Green LED ON
        }
else {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET); // Green LED OFF
        }
```

After saving the main.c file with the codes pasted into it by clicking the save button, the codes will look like Fig. 10.

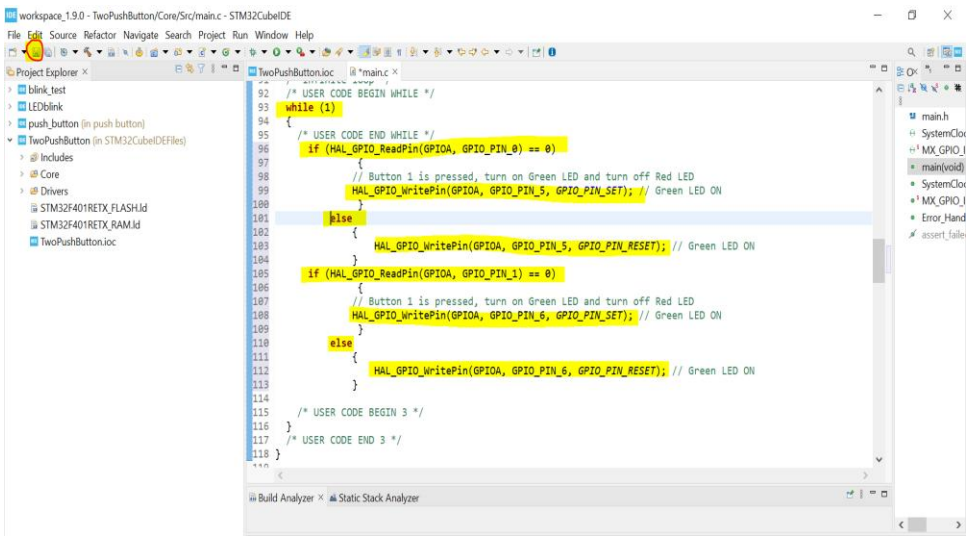

Fig. 10 Saving the code

Now, you need to generate the hex file. Click on Project >> Properties as shown in Fig. 11 and then a pop-up window will open as shown in Fig. 12 in which necessary configurations are required as shown in yellow highlights.

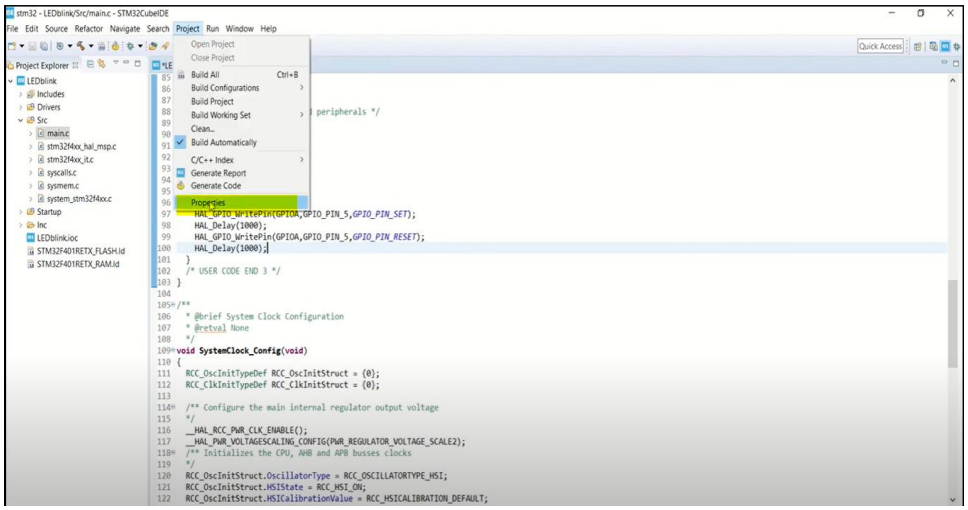

Fig. 11 Configuring for the HEX file generation

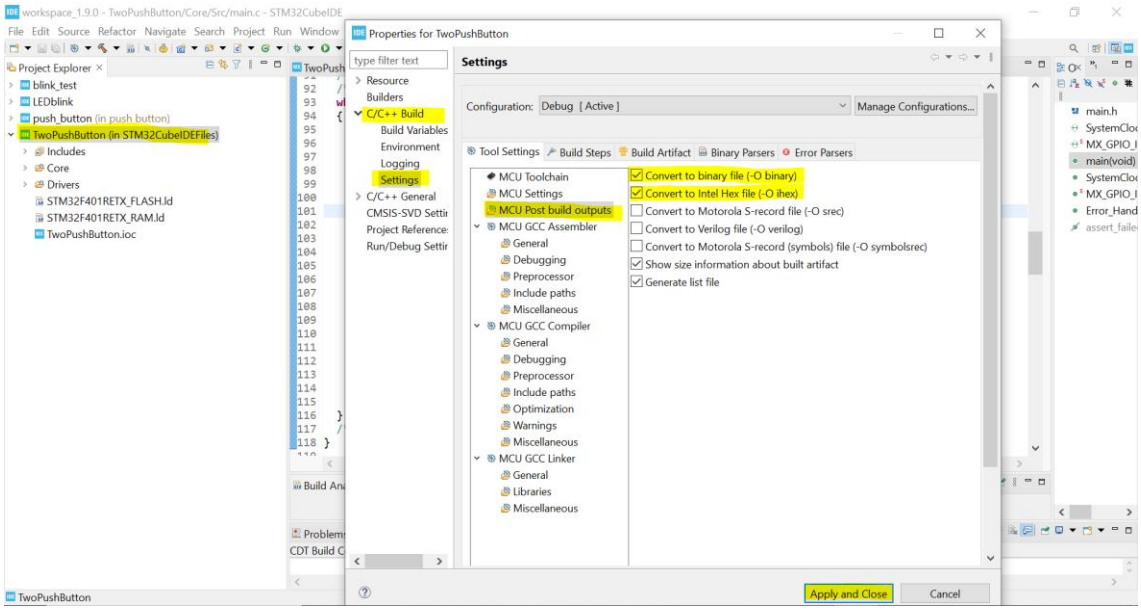

Fig. 12 Configuring for the HEX file generation

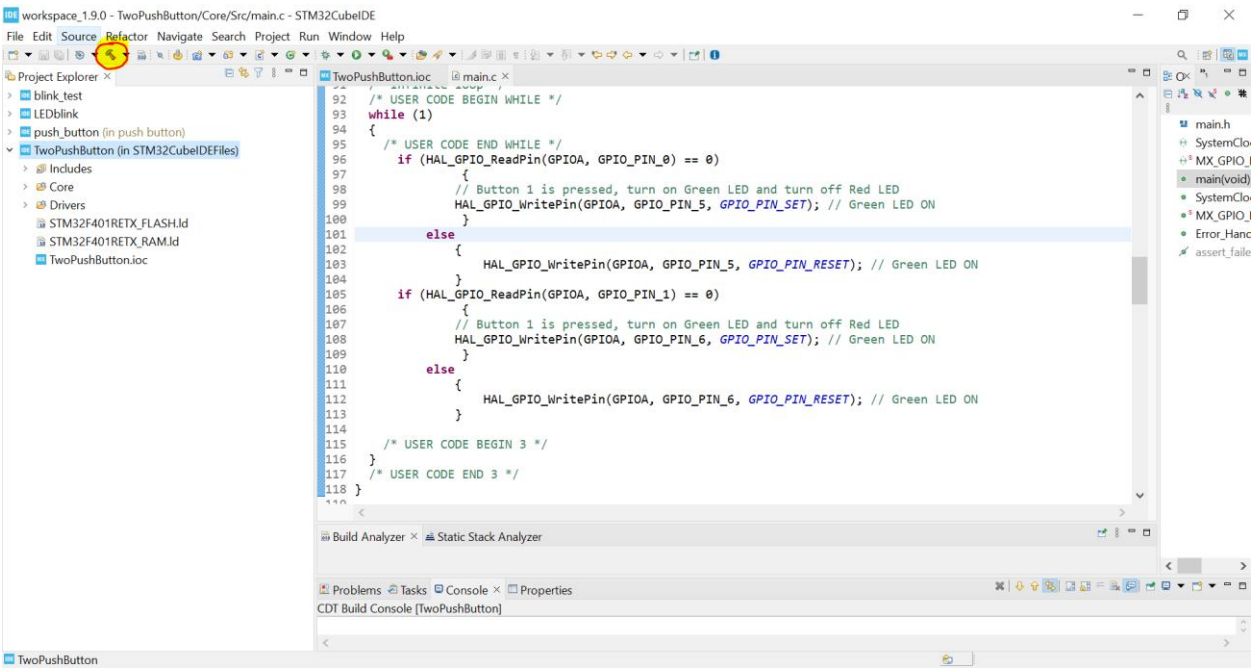

Fig. 13 Building the code and generating the HEX file.

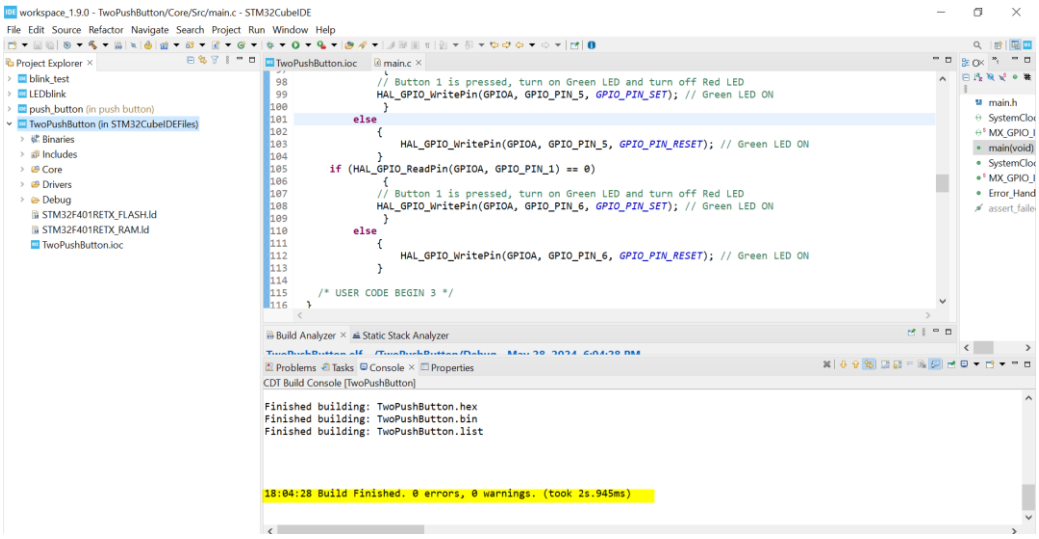

Fig. 14 Code building without any error and warning messages.

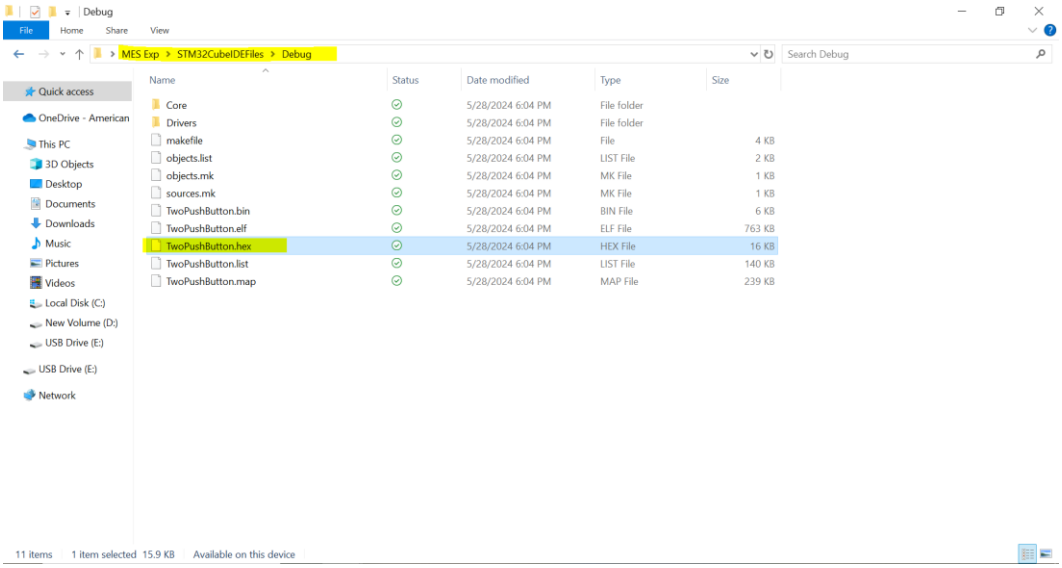

Fig. 15 The HEX file is created in the Debug folder of the saved location of your project

Steps for Proteus Implementation and Testing

Now you need to prepare the circuit diagram in the Proteus simulator. For this purpose, you need to follow the steps mentioned in the next figures to draw and simulate the circuit. First, Open the Proteus Design Suit from your PC or laptop.

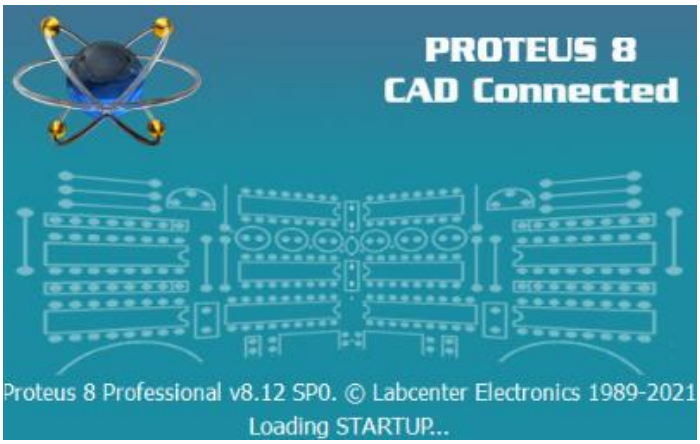

Fig. 16 Opening the design suit

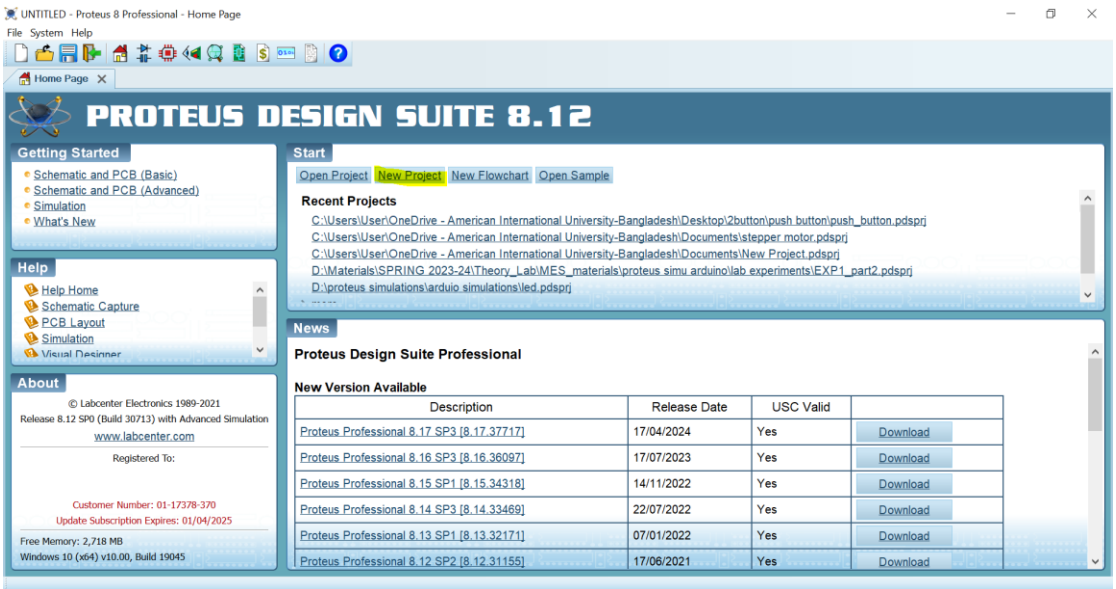

Fig. 17 Creating a new project

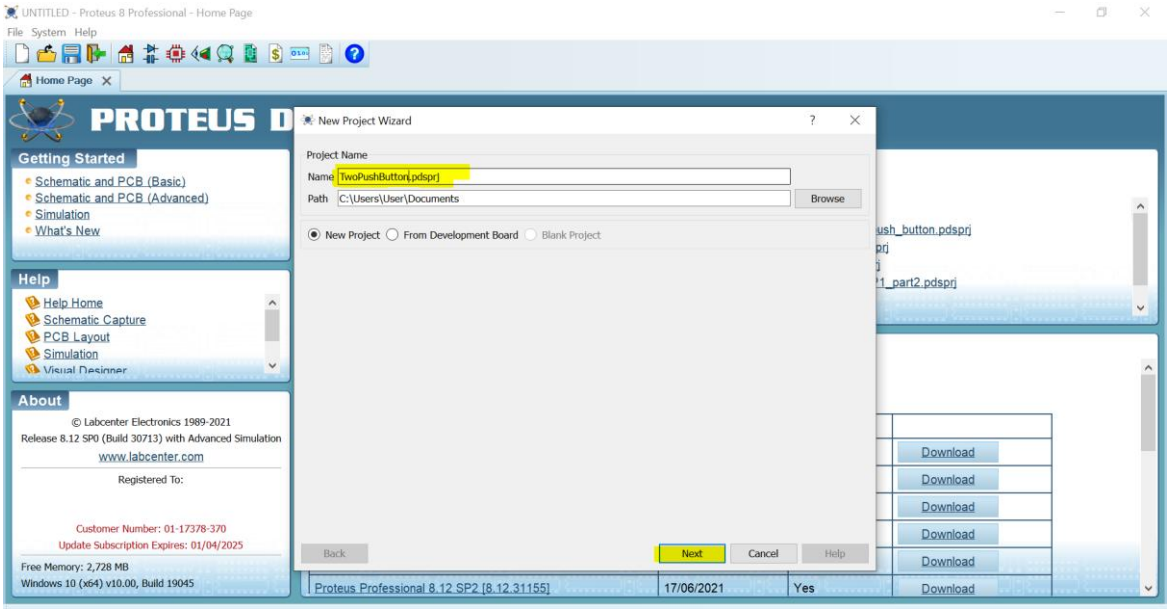

Fig. 18 Name the project

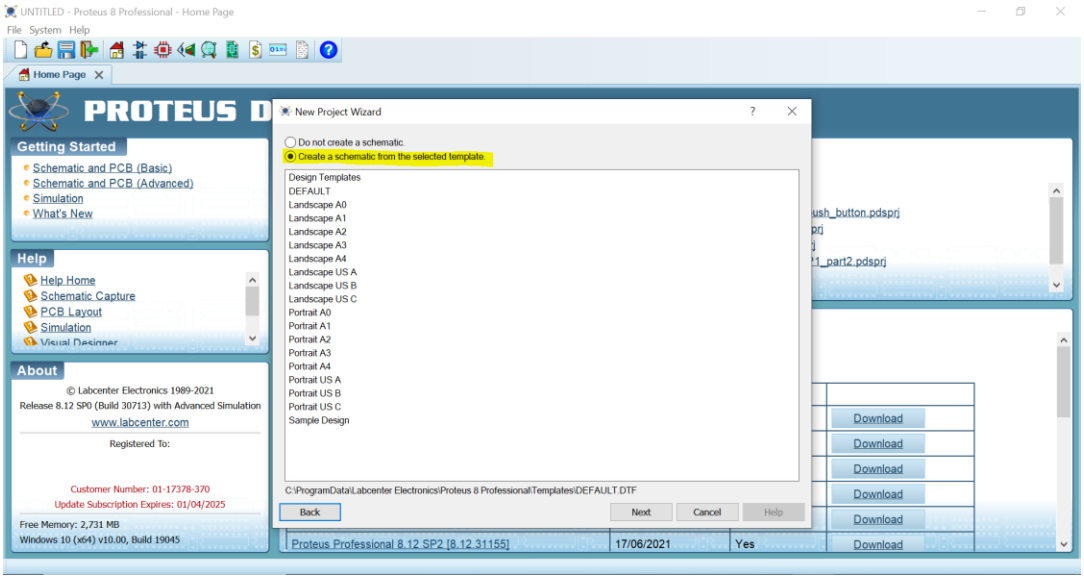

Fig. 19 Creating the schematic

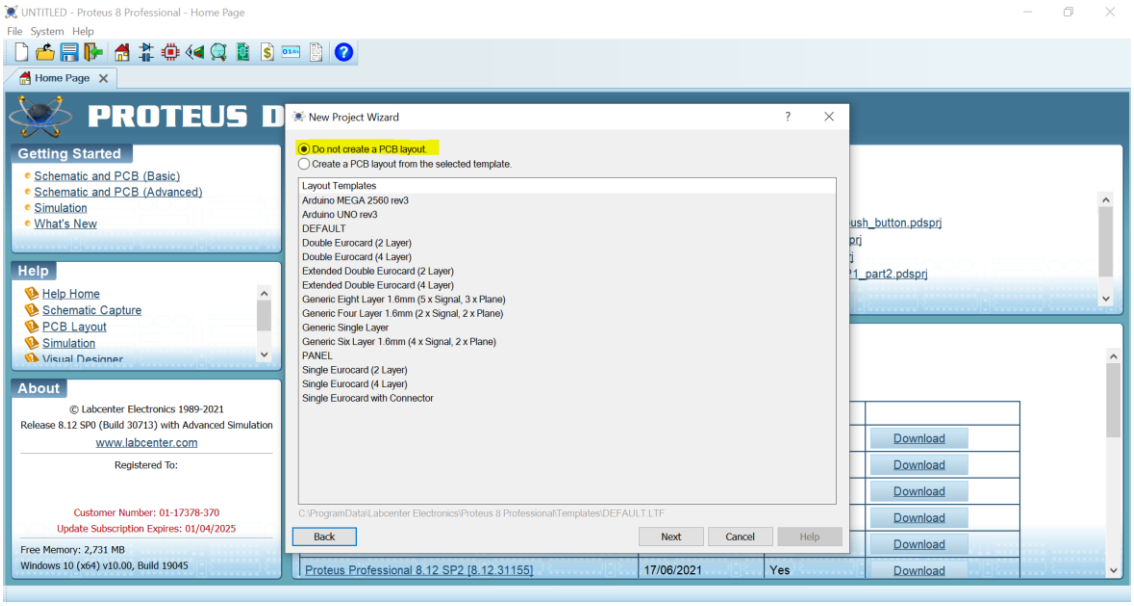

Fig. 20 Creating the schematic without PCB layout

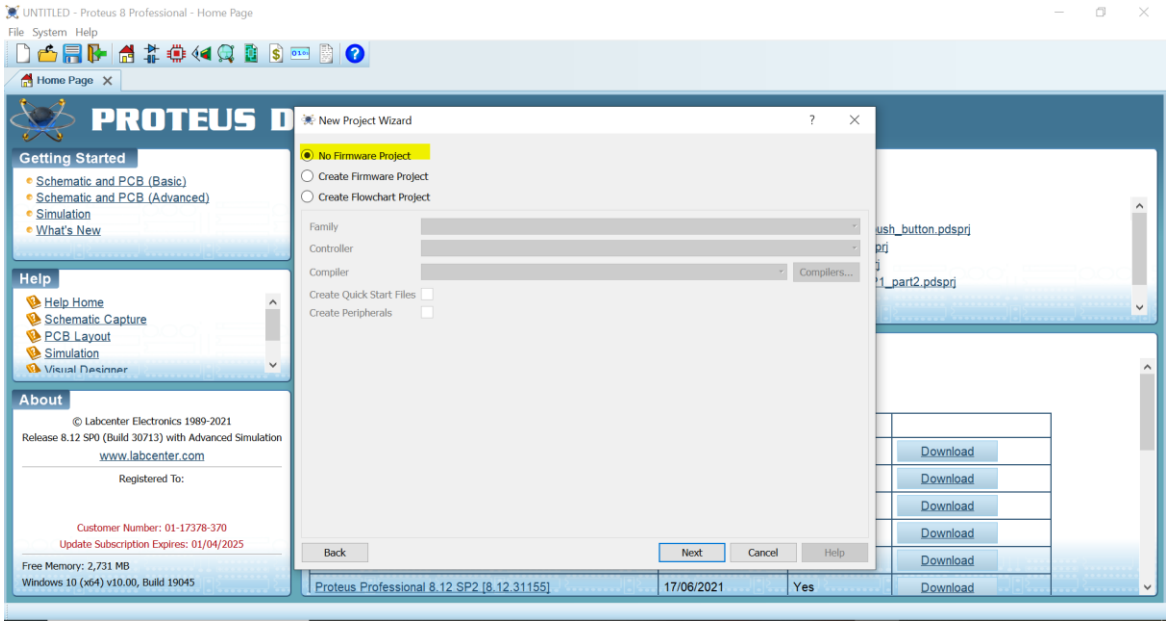

Fig. 21 Making no Firmware Project

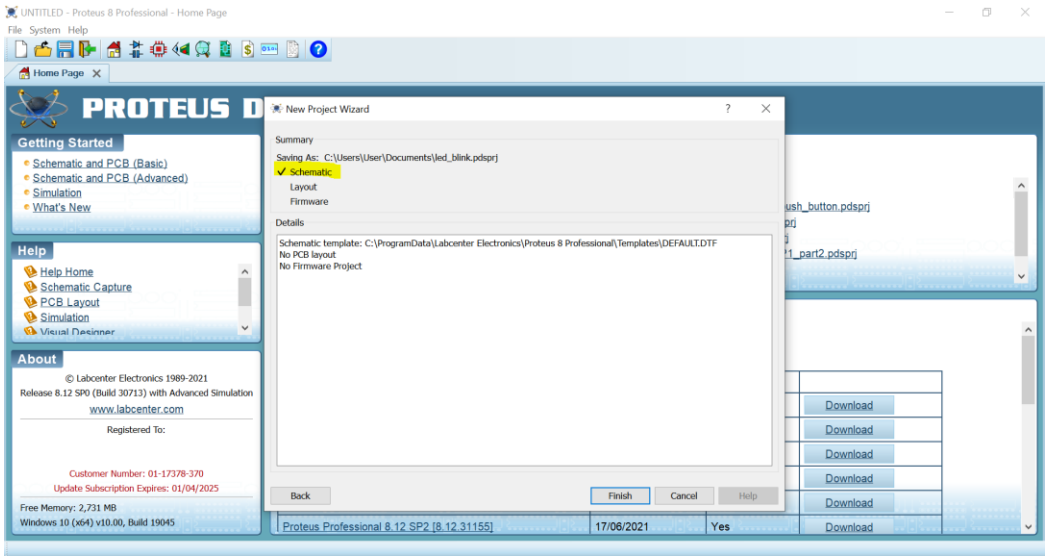

Fig. 22 Finish the initialization

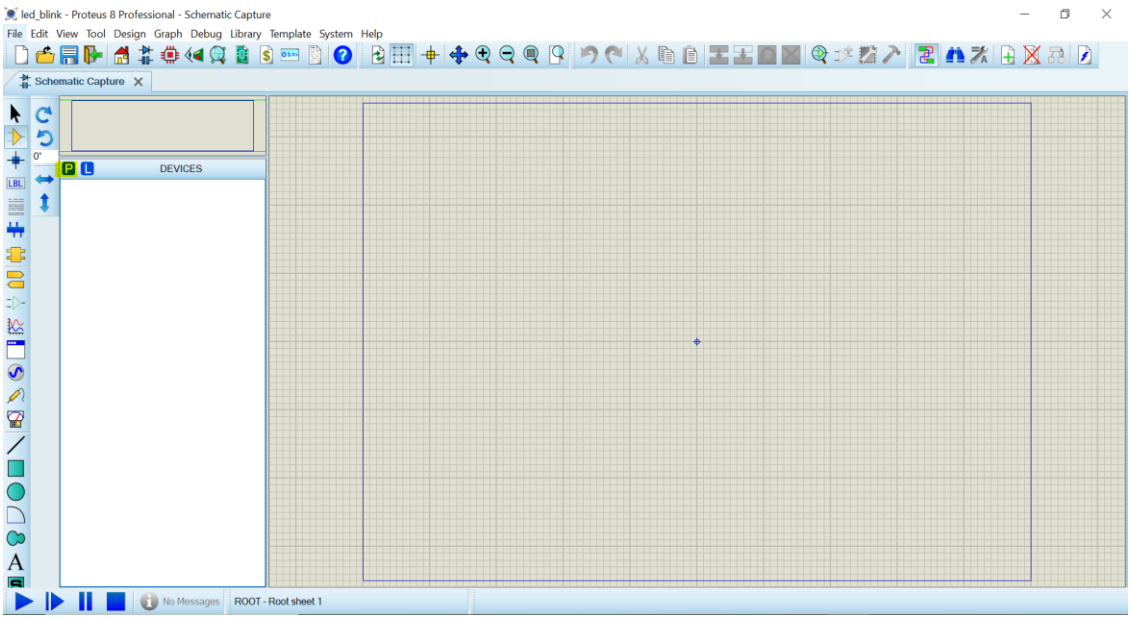

Fig. 23 Blank Workspace in Proteus Simulator

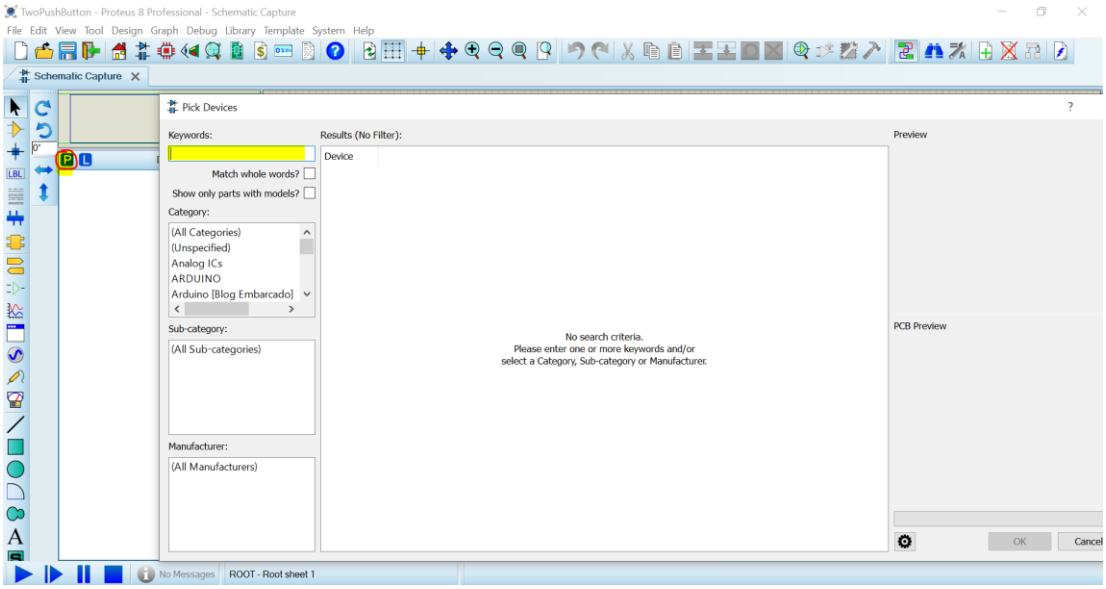

Fig. 24 Searching for the required Components 1

Click on icon P and search for the required component by typing the keywords into the search box.

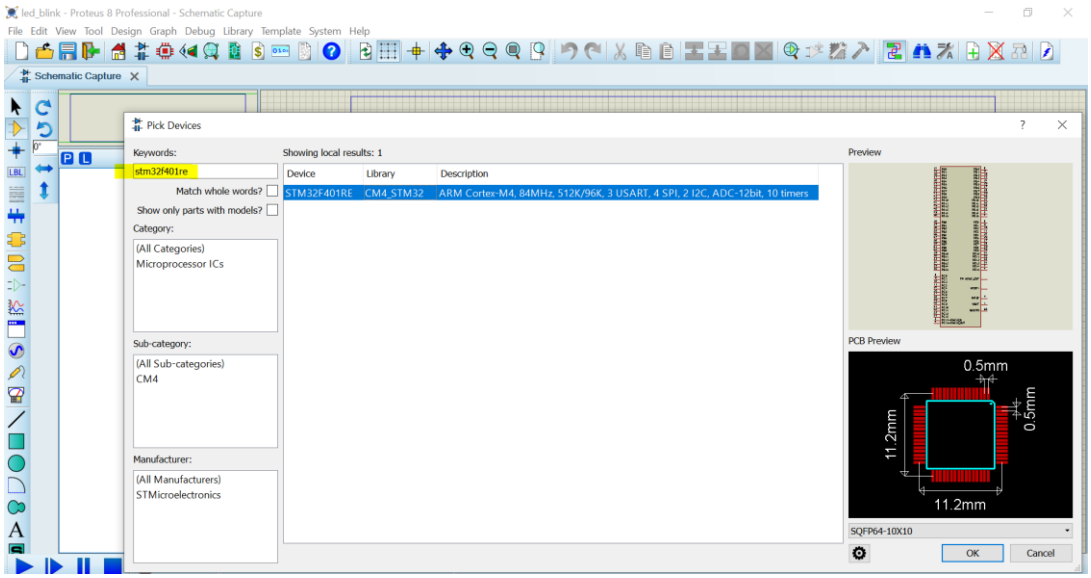

Fig. 25 Searching for the required Components 2

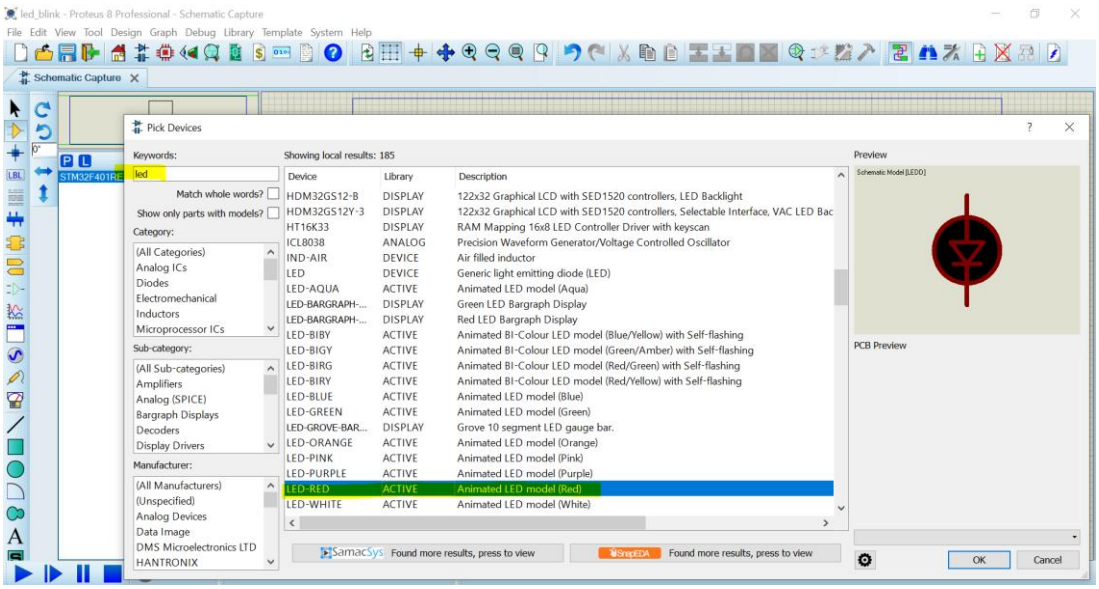

Fig. 26 Searching for the required Components 3

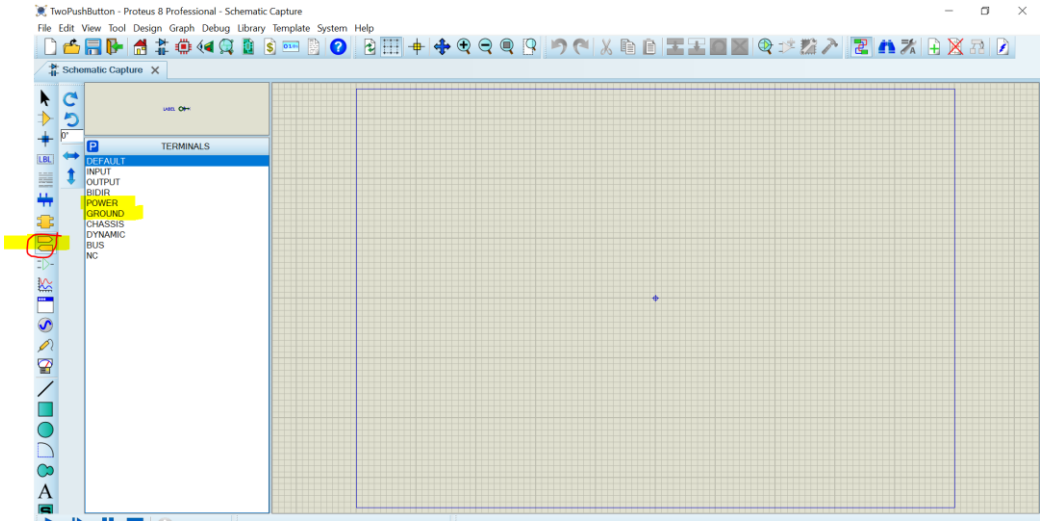

Fig. 27 Power and ground terminals can be found from here

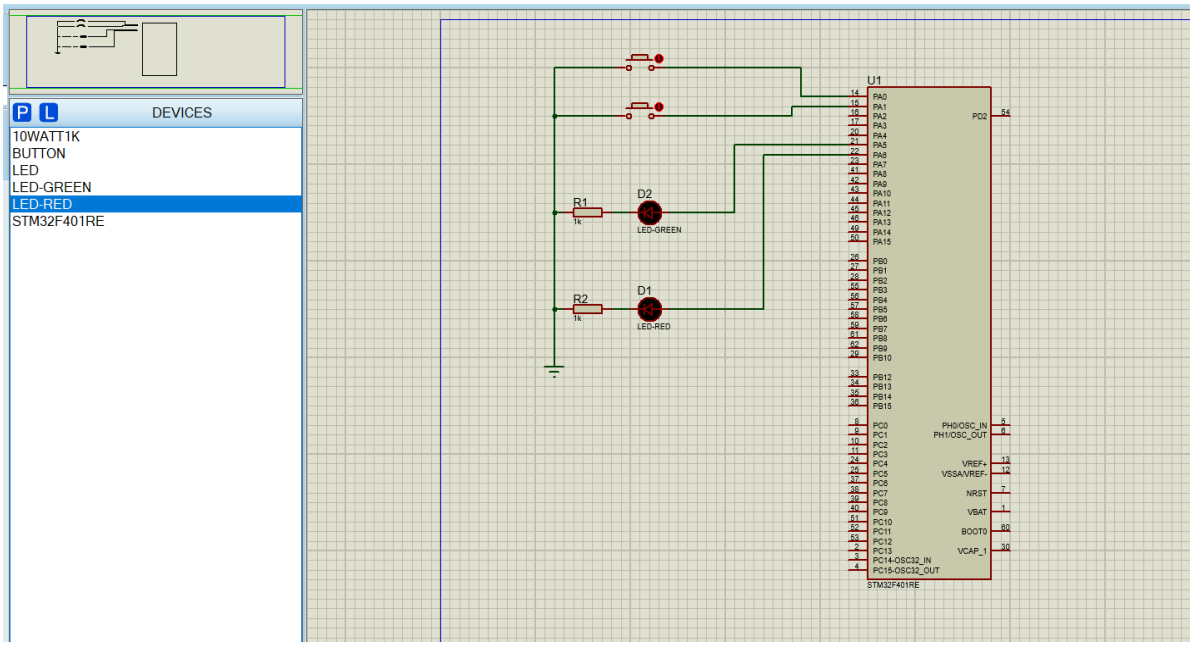

Fig. 28 Preparing the circuit of our 2 LED control system using 2 switches

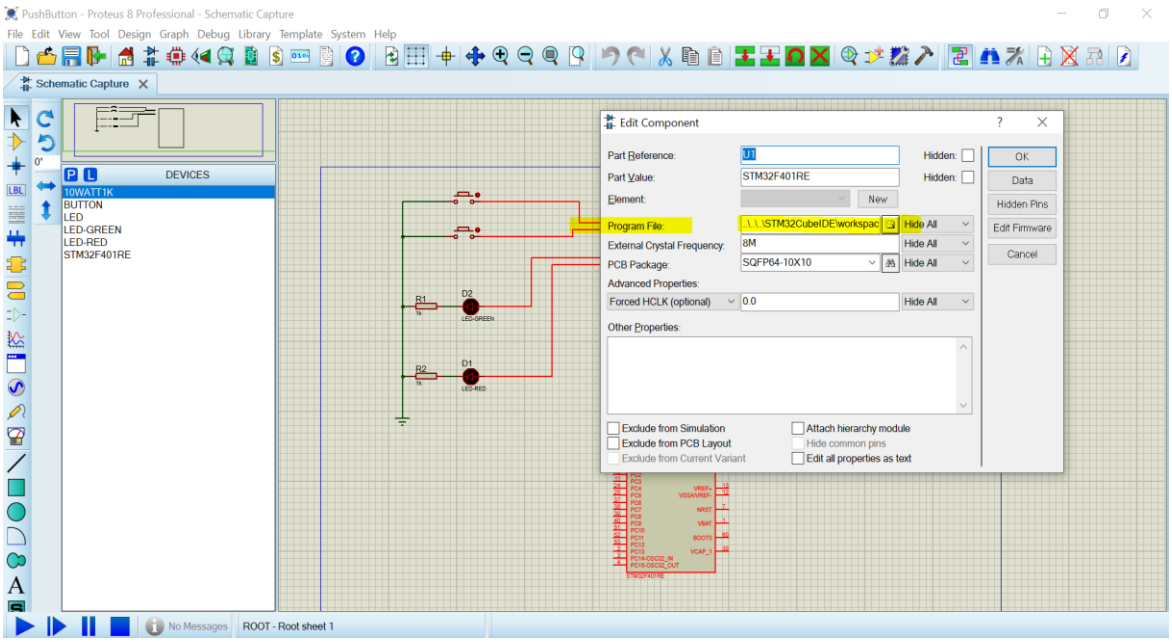

Fig. 29 Uploading the HEX file into the STM32F401RE Board

Double-click on the STM32F401RE board. In the program file box, select the HEX file from your saved location as shown on Fig. 15.

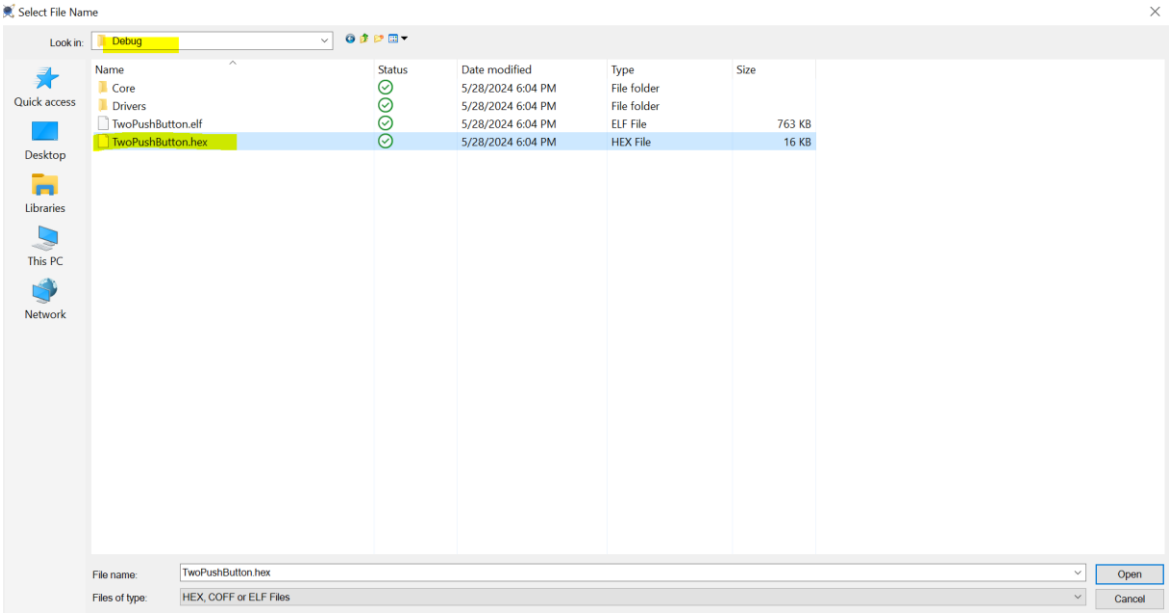

Fig. 30 Uploading the HEX file into the STM32F401RE Board

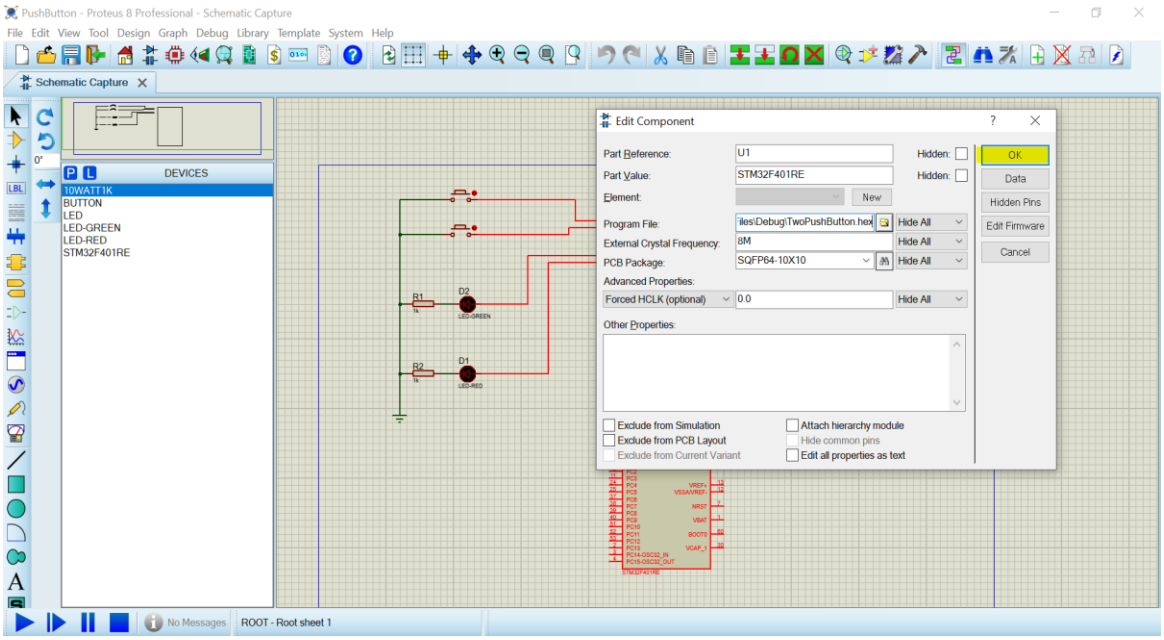

Fig. 31 Click OK and Save the workspace

Run the simulation using the Play button at the bottom-left corner as shown in Fig. 32.

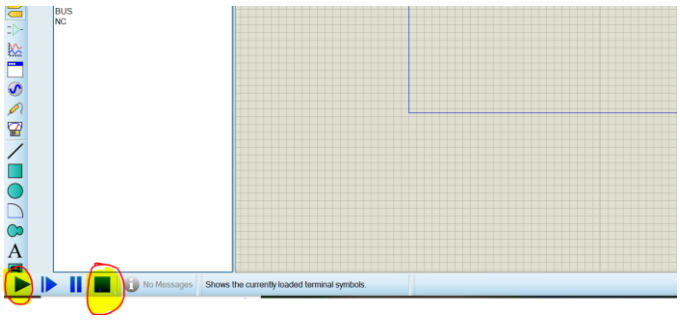

Fig. 32 Play and Stop Button

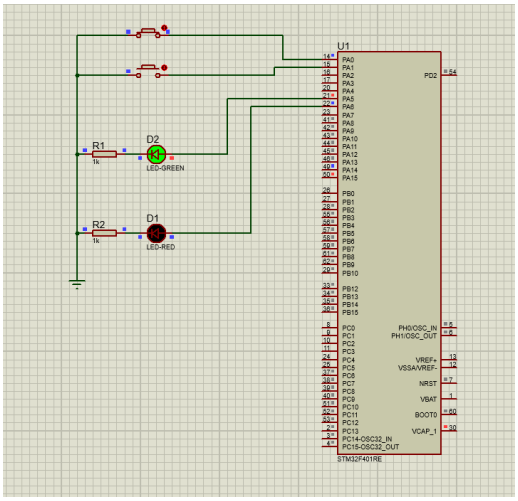

Fig. 33 Button1 is pressed, so the green LED is turned ON

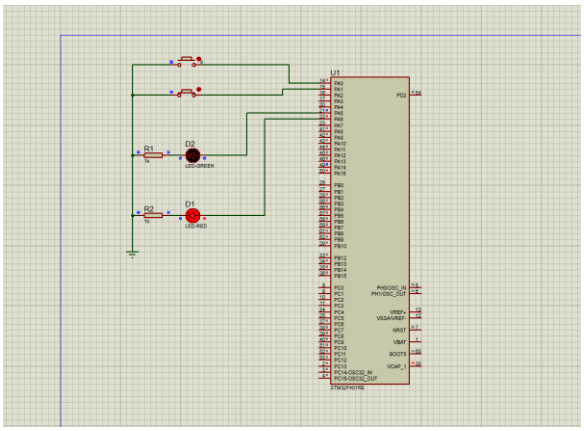

Fig. 38. Button2 is pressed, so the red LED is turned ON

After verifying the functionality of your developed system, now prepare the actual hardware using the STM32F401RE Board.

Questions for report writing:

1) Include all codes and scripts in the lab report following the lab report writing template.
2) Draw the **program flow chart** of both codes and explain them.
3) Include the **Proteus simulation** of the blink program and traffic light control system. You may learn the simulation from the following link: https://www.youtube.com/watch?v=MDsoLQicdAk.

Reference(s):

[1] https://www.st.com/en/evaluation-tools/nucleo-f401re.html for STM32F401RE datasheet.
[2] www.st.com.
[3] https://www.st.com/resource/en/user_manual/dm00105879-description-of-stm32f4-hal-and-ll-drivers-stmicroelectronics.pdf.
[4] www.st.com/en/development-tools/stm32cubeide.html.